Appendix C. A message-passing framework and complete ATM example

Back in chapter 4, I presented an example of sending messages between threads using a message-passing framework, using a simple implementation of the code in an ATM as an example. What follows is the complete code for this example, including the message-passing framework.

Listing C.1 shows the message queue. It stores a list of messages as pointers to a base class; the specific message type is handled with a template class derived from that base class. Pushing an entry constructs an appropriate instance of the wrapper class and stores a pointer to it; popping an entry returns that pointer. Because the `message_base` class doesn't have any member functions, the popping thread will need to cast the pointer to a suitable `wrapped_message<T>` pointer before it can access the stored message.

Listing C.1. A simple message queue

```
#include <mutex>
#include <condition_variable>
#include <queue>
#include <memory>

namespace messaging
{
    struct message_base          <-- Base class of our queue entries
    {
        virtual ~message_base()
        {}
    };
    template<typename Msg>
    struct wrapped_message:      <-- Each message type has a specialization
        message_base
    {
        Msg contents;

        explicit wrapped_message(Msg const& contents_):
            contents(contents_)
```